

**``LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 3
DESIGN PRINCIPLE (S.O.L.I.D)**

DISUSUN OLEH :

DANDI TAUFIK HIDAYAT

NIM : 2250081079



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2024**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR	ii
BAB I HASIL PRAKTIKUM	3
I.1 Latihan ISP	3
I.1.A. Source Code	3
I.1.B. Screenshot	5
I.1.C. Analisis	5
I.2 Latihan DPI	6
I.2.A. Source Code	6
I.2.B. Screenshot	7
I.2.B. Analisis	7
BAB II TUGAS PRAKTIKUM.....	8
II.1 Tugas 1	8
II.1.A. Analisis	8
II.2 Tugas 2	8
II.1.A. Source Code	8
II.1.B. Screenshot.....	8
II.1.C. Analisis	9
II.3 Tugas 3.3.....	9
II.1.A. Analisis	9
BAB III KESIMPULAN.....	10

DAFTAR GAMBAR

<i>Gambar 1 Tanpa ISP</i>	<i>5</i>
<i>Gambar 2 Dengan ISP.....</i>	<i>5</i>
<i>Gambar 3 Tanpa DIP</i>	<i>7</i>
<i>Gambar 4 Dengan DIP</i>	<i>7</i>
<i>Gambar 5 Code Motorcycle.....</i>	<i>8</i>

BAB I

HASIL PRAKTIKUM

I.1 Latihan ISP

I.1.A. Source Code

a) Tanpa Penerapan ISP

```
interface VehicleInterface {  
    void drive();  
    void stop();  
    void refuel();  
    void openDoors();  
}
```

```
class Motorcycle implements VehicleInterface {  
    // Methods that can be implemented  
    @Override  
    public void drive() {  
        // Implementation of driving logic for motorcycle  
    }  
  
    @Override  
    public void stop() {  
        // Implementation of stopping logic for motorcycle  
    }  
  
    @Override  
    public void refuel() {  
        // Implementation of refueling logic for motorcycle  
    }  
  
    // Method that cannot be implemented for a motorcycle  
    @Override  
    public void openDoors() {  
        throw new  
        UnsupportedOperationException("Motorcycles do not have  
        doors.");  
    }  
}
```

```
class Car implements VehicleInterface {  
    // Methods that can be implemented  
    @Override  
    public void drive() {  
        // Implementation of driving logic for car  
    }  
  
    @Override  
    public void stop() {  
        // Implementation of stopping logic for car  
    }  
}
```

```

        @Override
        public void refuel() {
            // Implementation of refueling logic for car
        }

        @Override
        public void openDoors() {
            // Implementation of opening doors logic for car
        }
    }

```

b) Dengan menerapkan Interface Segregation Principle

```

interface VehicleInterface {
    void drive();
    void stop();
    void refuel();
    void openDoors();
}

```

```

class Motorcycle implements VehicleInterface {
    // Methods that can be implemented
    @Override
    public void drive() {
        // Implementation of driving logic for motorcycle
    }

    @Override
    public void stop() {
        // Implementation of stopping logic for motorcycle
    }

    @Override
    public void refuel() {
        // Implementation of refueling logic for motorcycle
    }

    // Method that cannot be implemented for a motorcycle
    @Override
    public void openDoors() {
        throw new
        UnsupportedOperationException("Motorcycles do not have
        doors.");
    }
}

```

```

class Car implements VehicleInterface {
    // Methods that can be implemented
    @Override
    public void drive() {
        // Implementation of driving logic for car
    }

    @Override
    public void stop() {
    }
}

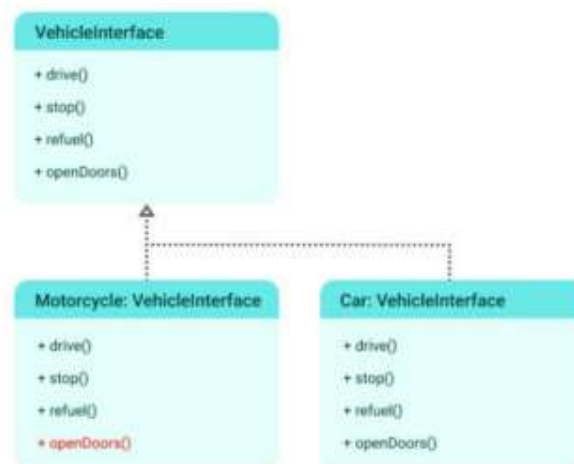
```

```

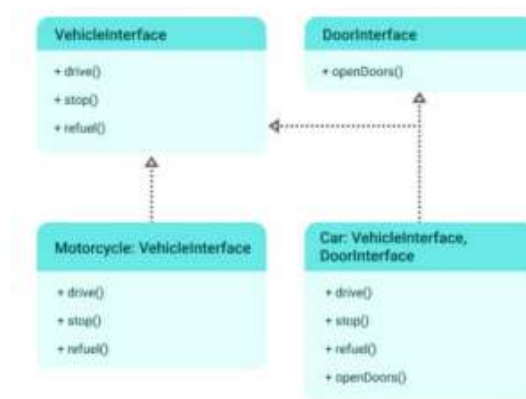
@Override
public void refuel() {
    // Implementation of refueling logic for car
}
@Override
public void openDoors() {
    // Implementation of opening doors logic for car
}
}

```

I.1.B. Screenshot



Gambar 1 Tanpa ISP



Gambar 2 Dengan ISP

I.1.C. Analisis

Pada Latihan 1 ini terdapat 2 contoh source code yang memiliki perbedaan dalam mengimplementasi codenya, yaitu tanpa penerapan ISP (Interface Segregation Principle) dan menggunakan ISP. Lebih dari ISP ini, class-class yang saling bergantung dapat berkomunikasi dengan menggunakan interface yang lebih kecil, mengurangi ketergantungan pada fungsi-fungsi yang tidak digunakan dan mengurangi coupling.

I.2 Latihan DPI

I.2.A. Source Code

- a) Tanpa menggunakan prinsip Dependency Inversion Principle

```
class Car {
    private final Engine engine;
    public Car(Engine engine) {
        this.engine = engine;
    }
    void start() {
        engine.start();
    }
}
```

```
class Engine {
    void start() {
    }
}
```

```
class DieselEngine {
    void start() {
    }
}
```

- b) Menggunakan prinsip Dependency Inversion Principle

```
class PetrolEngine implements EngineInterface {
    @Override
    public void start() {
    }
}
```

```
class HybridEngine implements EngineInterface {
    @Override
    public void start() {
    }
}
```

```
class DieselEngine implements EngineInterface {
    @Override
    public void start() {
    }
}
```

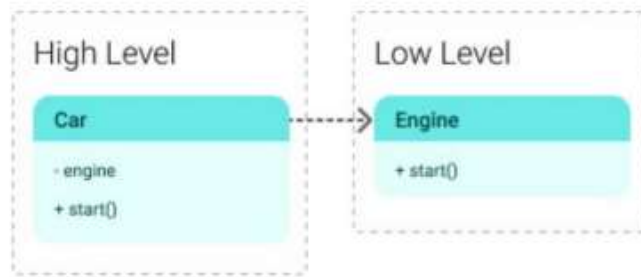
```
public class Main {
    public static void main(String[] args) {
        // Instantiate cars with different engine types
        Car fuelCar = new Car(new PetrolEngine());
        Car dieselCar = new Car(new DieselEngine());
        Car hybridCar = new Car(new HybridEngine());
        // Start each car
        fuelCar.start();
        dieselCar.start();
    }
}
```

```

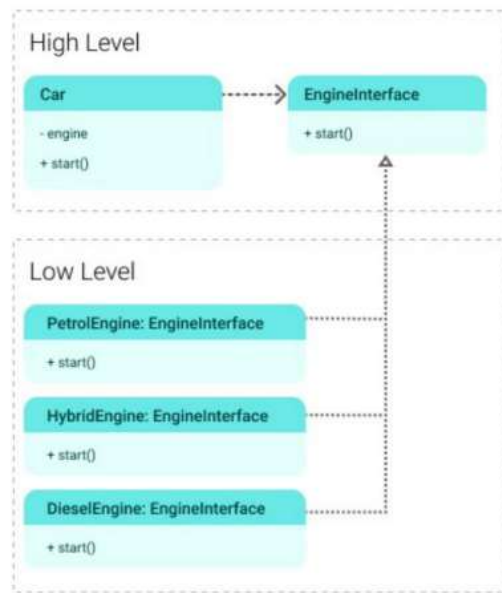
        hybridCar.start();
    }
}

```

I.2.B. Screenshot



Gambar 3 Tanpa DIP



Gambar 4 Dengan DIP

I.2.B. Analisis

Pada latihan ke 2 ini terdapat 2 source code ini membuat sebuah class car yang didalamnya terdapat class engine dalam penerapan tanpa Dependency Inversion Principle (DIP) kemudian dalam penerapan dengan menggunakan prinsip DIP terdapat beberapa class yaitu class petrolengine, hybridengine, dan dieselengine. Prinsip DIP ini hampir sama dengan konsep layering dalam aplikasi, di mana low-level modules bertanggung jawab dengan fungsi yang sangat detail dan high-level modules menggunakan low-level classes untuk mencapai tugas yang lebih besar

BAB II

TUGAS PRAKTIKUM

II.1 Tugas 1

II.1.A. Analisis

Pada tugas ke 1 ini kenapa bentuk kode nya harus di ubah agar mempermudah pemeliharaan, pengujian dan pengembangan selanjutnya, memenuhi prinsip ISP itu sendiri yaitu membuat kode lebih modular dan relevan untuk setiap kelas. Dengan desain yang lebih modular, kita memastikan bahwa setiap kelas hanya mengimplementasikan metode yang benar-benar dibutuhkan, meningkatkan efisiensi serta kualitas desain kode.

II.2 Tugas 2

II.1.A. Source Code

```
class Motorcycle implements VehicleInterface {
    @Override
    public void drive() {
        System.out.println("Motorcycle is driving...");
    }

    @Override
    public void stop() {
        System.out.println("Motorcycle has stopped.");
    }

    @Override
    public void refuel() {
        System.out.println("Motorcycle is refueling...");
    }
}
```

II.1.B. Screenshot



Gambar 5 Code Motorcycle

II.1.C. Analisis

Pada tugas ini melengkapi class motorcycle dengan menambahkan keluaran ketika kondisi drive dia akan memunculkan Motorcycle is driving, Stop maka keluar Motorcycle is stopped, dan ketika Refuel maka akan mengembalikan keluaran Motorcycle is refueling, namun karena tidak main pada package tersebut code nya tidak bisa dijalankan, karena hanya konsep dari penerapan LSP.

II.3 Tugas 3.3

II.1.A. Analisis

Kode harus diubah karena untuk menerapkan konsep dari dependency inversion principle DIP, kemudian memudahkan untuk penambahan fitur ketika penambahan fitur baru tanpa memodifikasi kode yang ada dan mempermudah pengujian dan pemeliharaan. Dengan desain ini, dapat menciptakan sistem yang lebih stabil, modular, dan dapat berkembang seiring kebutuhan.

BAB III

KESIMPULAN

Pada pertemuan ke 3 dimata kuliah praktikum Implementasi Perangkat Lunak ini melanjutkan dari modul kemarin yaitu penerapan SOLID. Dilanjutkan mulai dari ISP (Interface Segregation Principle) ini bertujuan untuk mengurangi jumlah ketergantungan sebuah class terhadap interface class yang tidak dibutuhkan. Jumlah ketergantungan dari fungsi pada sebuah interface class yang dapat diakses oleh class tersebut harus dioptimalkan atau dikurangi, kemudian DIP (Dependency Inversion hampir sama dengan konsep layering dalam aplikasi, di mana low-level modules bertanggung jawab dengan fungsi yang sangat detail dan high-level modules menggunakan low-level classes untuk mencapai tugas yang lebih besar